UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ingeniería Industrial y de Sistemas · Escuela de Ingeniería de Software

GUÍA PRÁCTICA — SEMANA 10

Sprint 1 e integración continua con GitHub Actions

SW705 — Taller de Ingeniería de Software I

Docente: Mg. Ing. José Carlos, García La Riva

Caso aplicado: Clínica San Rafael S.A.C.

Integrantes del equipo	Albino Soto, Christopher Henrry · Inga Champi, Franz Joe · Rashuaman Sapallanay, Ricco Didier
Código(s)	20231158C · 20231302G · 20231242D
Sección	SW705-U · Grupo A
Fecha	9 de noviembre de 2026
Repositorio del equipo	https://github.com/francito69/SW705U-grupo-A

📌 Estado de esta entrega (corte: 9 de noviembre de 2026)

Hecho en GitHub: pull request #5 (https://github.com/francito69/SW705U-grupo-A/pull/5) con la estructura base del backend y el flujo de CI. GitHub Actions corrió 3 veces: verde → rojo provocado → verde (partes 2, 3 y 4).

Hecho y listo para subir: HU03 con la prueba escrita primero (commits 42474ea y 6ff250b) y la insignia del README (fcb6a97), en ramas que parten del PR #5 (partes 6 y 7). Todas las salidas de consola son reales; las largas se recortan y se indica dónde.

PENDIENTE: aprobación y fusión del PR #5, regla de protección de main (Parte 5) y, después, los pull requests de HU03 y de la insignia con sus ejecuciones en Actions. Cada sección lo indica.

Instrucciones generales

En esta práctica el equipo avanza el Sprint 1 de su proyecto integrador: construye una historia con su prueba automática, activa la integración continua con GitHub Actions, protege la rama principal y deja registrado el incremento. Los ejemplos usan el caso Clínica San Rafael S.A.C. (FastAPI, pytest); si su proyecto usa otra tecnología, adapte los comandos del flujo y conserve el mismo orden de trabajo.

Modalidad y evaluación

Aspecto	Detalle
Modalidad	En equipo, con trabajo individual dentro del equipo: cada integrante construye su historia en su propia rama y el equipo comparte el flujo de integración continua.
Duración	4 horas de taller (Bloques: 0.5 h Daily y tablero, 1.5 h historia, 1 h integración continua, 1 h pull request, protección y registro).
Requisitos	Repositorio de la Semana 9 con README, .gitignore y rama principal; Python 3.12 o superior; Git; cuenta de GitHub de cada integrante.
Evaluación	Formativa: el docente retroalimenta con la rúbrica del final de este documento. No promedia en las prácticas: la calificación sumativa del Avance 5 es en la Semana 11.
Entregable	Repositorio con el flujo de CI activo, pull request de la historia con el check en verde, rama principal protegida y este documento completado con evidencias.

🧭 Reglas del trabajo

Ninguna historia llega a la rama principal sin pull request, sin el check de CI en verde y sin la aprobación de otro integrante.

Nunca se suben datos reales de pacientes, contraseñas ni claves: solo datos ficticios en pruebas y ejemplos.

Si el build se pone rojo, se lee el registro antes de pedir ayuda; el mensaje de error casi siempre indica el archivo y la línea.

Cada evidencia debe poder verificarse en el repositorio (enlace a la ejecución, al pull request o al commit).

Parte 1 — La jornada de rodaje: tablero y Definición de Hecho

Antes de escribir código, el equipo revisa el tablero del Sprint 1 y confirma quién hace qué hoy. Luego actualiza la Definición de Hecho (DoD) acordada en la Semana 7 con una condición nueva: el build de integración continua en verde.

1.1 Asignación de la jornada

Integrante	Historia o tarea de hoy	Puntos	Rama	Estado al iniciar
Albino Soto, Christopher Henrry	HU03 Registro y búsqueda de paciente: estructura base del backend y endpoints GET y POST /pacientes, con la prueba escrita primero	3	feature/HU03-registro-paciente (la estructura base va en el PR #5)	Por hacer
Albino Soto, Christopher Henrry	Tarea de HU03: flujo de GitHub Actions (ci.yml), registro de builds e insignia del README	(en HU03)	ci/github-actions (PR #5) y docs/insignia-ci	Por hacer
Rashuaman Sapallanay, Ricco Didier	HU02 Agendamiento de citas sin solapamientos (POST /citas con 409 ante un cruce). Revisa y aprueba los pull requests del flujo y de HU03	5	feature/HU02-agendamiento-citas	Por hacer
Inga Champi, Franz Joe	HU01b Acceso por rol y bitácora: inicio de sesión con JWT y verificación de rol. Como dueño del repositorio, fusiona y crea la regla de protección de main	3	feature/HU01b-acceso-rol	Por hacer

Cada fila debe corresponder a una historia del Sprint Backlog de la Semana 7 y a una rama con nombre según la convención de la Semana 9.

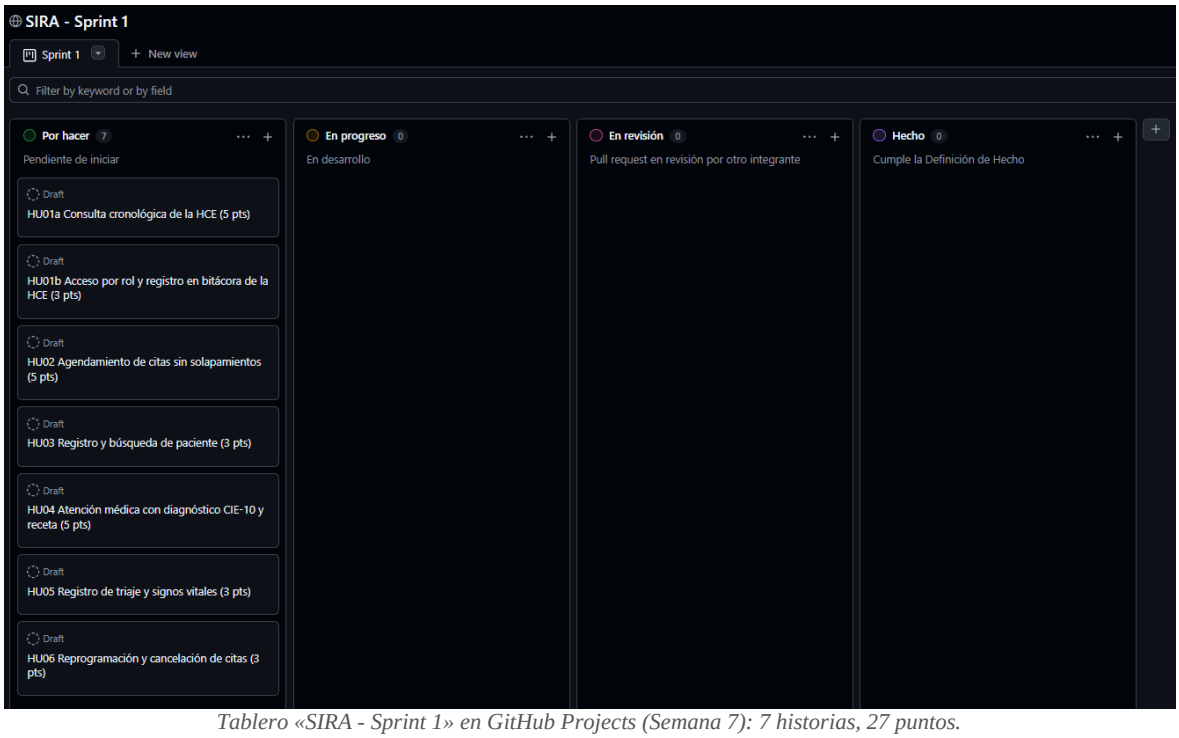
1.2 Definición de Hecho con integración continua

N.º	Criterio verificable de la Definición de Hecho	¿Cómo se verifica?
1	El código está integrado en main mediante un pull request aprobado por un integrante distinto del autor.	Pestaña Reviews del pull request con la aprobación.
2	Las pruebas de la historia pasan y la cobertura de app/ es de 80 % o más.	pytest --cov=app --cov-fail-under=80 (backend); Vitest (frontend).
3	Cada escenario Dado/Cuando/Entonces de la historia tiene al menos una prueba automática que lo verifica.	Tabla 6.1 y nombres de las pruebas en tests/.
4	**NUEVO —** El build de integración continua está en verde: el check test (instalación, ruff y pytest con cobertura mínima) pasa en el pull request antes de fusionar. Reemplaza a flake8 de la Semana 7.	Check test en verde en el pull request y enlace a la ejecución en Actions.
5	Cada endpoint nuevo está documentado con sus códigos de respuesta.	Swagger de FastAPI en /docs.

N.º	Criterio verificable de la Definición de Hecho	¿Cómo se verifica?
6	Las operaciones sobre datos clínicos validan el rol (403), quedan en bitacora_auditoria y no usan borrado físico.	Prueba con un rol no autorizado y registro en la bitácora.
7	No hay errores abiertos de prioridad alta y el Product Owner aceptó la historia en la Sprint Review.	Tablero sin errores de prioridad alta; tarjeta en Hecho con la aprobación del PO.

Evidencia: captura del tablero con las historias del Sprint 1 y de la tabla anterior completada.

Evidencia del equipo: tabla 1.1 y Definición de Hecho completadas. La captura del tablero es la registrada en la Semana 7 (Entregable 5), con las 7 historias en «Por hacer». Captura actualizada con HU03 en «En revisión»: PENDIENTE (mover la tarjeta en el tablero y tomar la captura).



Tablero «SIRA - Sprint 1» en GitHub Projects (Semana 7): 7 historias, 27 puntos.

Parte 2 — Punto de partida: ejecutar el proyecto en su equipo

Antes de automatizar, cada integrante comprueba que el proyecto funciona en su computadora. Lo que no corre en su equipo no puede correr en el servidor de integración.

N.º	Qué hacer	Comando o acción

2.1	Clonar el repositorio del equipo (o traer los últimos cambios si ya lo tiene) y entrar a la carpeta.	`git clone https://github.com/USUARIO/REPO.git`
		`cd REPO`
		`git switch main`
		`git pull`

2.2	Crear y activar un entorno virtual. En Windows se activa con venv\Scripts\activate.	`python3 -m venv venv` `source venv/bin/activate`
2.3	Instalar las dependencias del proyecto.	`pip install -r requirements.txt`
2.4	Ejecutar las pruebas existentes y anotar cuántas pasan.	`pytest -q`
2.5	Levantar la aplicación y abrir la documentación automática en el navegador (detener con Ctrl + C).	`uvicorn app.main:app --reload` Abrir http://127.0.0.1:8000/docs

Dato	Resultado en su equipo
Versiones	Python 3.12.3 · git 2.43.0 (clon desde GitHub; rama ci/github-actions del PR #5, con la estructura base en baa74cd).
Pruebas existentes (pytest -q)	**2 passed in 1.06s** tests/test_base.py: GET /salud y GET /convenios. Antes de la estructura base, el corte de la mañana (equipo de Inga Champi) daba «no tests ran» (0 pruebas).
Aplicación y página /docs	uvicorn app.main:app --reload levanta la API en http://127.0.0.1:8000. /docs muestra GET /convenios y GET /salud; GET /salud responde {"estado": "ok"}.

Evidencia: captura de pytest -q con el resultado y de la página /docs.

Evidencia del equipo: salida real de los pasos 2.1 a 2.4 en un clon desde GitHub y captura de /docs con la aplicación en ejecución.



Pasos 2.1 a 2.4: clon desde GitHub, entorno virtual, instalación y pytest -q (2 passed).



Paso 2.5: documentación automática en http://127.0.0.1:8000/docs.

Parte 3 — Activar la integración continua con GitHub Actions

La tarea de esta parte es de equipo: la ejecuta una persona (en la solución, Ana, mantenedora del repositorio) y las demás la revisan. El flujo se escribe en un archivo del repositorio y GitHub lo ejecuta en cada push y en cada pull request hacia la rama principal.

🧭 Qué hace cada línea del flujo

on: define cuándo se ejecuta (push a main y pull requests hacia main).

jobs / runs-on: define un trabajo llamado test que corre en una máquina Ubuntu de GitHub.

steps: pasos en orden: descargar el código, preparar Python, instalar dependencias, revisar el estilo y ejecutar las pruebas. Si un paso falla, los siguientes no se ejecutan y el check queda en rojo.

Complete los espacios _____ del archivo de la Parte 3.3 usando esta explicación.

N.º	Qué hacer	Comando o acción

3.1	Partir de la rama principal actualizada y crear la rama de trabajo.	`git switch main`
		`git pull`
		`git switch -c ci/github-actions`

3.2	Crear el archivo requirements-dev.txt con las herramientas de desarrollo (no se instalan en producción).	`ruff==0.17.4` `pytest-cov==7.1.0`
3.3	Crear la carpeta .github/workflows y dentro el archivo ci.yml con el contenido del cuadro de la página siguiente.	`mkdir -p .github/workflows`

3.4	Registrar el cambio y subir la rama. No ejecute ruff en su equipo todavía: queremos ver qué detecta el servidor.	`git add .github requirements-dev.txt`
		`git commit -m "ci: agrega flujo de GitHub Actions con ruff y pytest"`
		`git push -u origin ci/github-actions`

3.5	En GitHub, abrir un pull request de ci/github-actions hacia main y abrir la pestaña Checks (o Actions) para ver la ejecución.	GitHub → Pull requests → New pull request

En nuestro caso: rama ci/github-actions creada desde main con la estructura base del backend (baa74cd) y el commit 951c2d4 «ci: agrega flujo de GitHub Actions con ruff y pytest»; requirements-dev.txt con ruff==0.17.4 y pytest-cov==7.1.0; pull request #5: https://github.com/francito69/SW705U-grupo-A/pull/5. También se agregó el bloque permissions: contents: read que recomienda el recuadro «Buenas prácticas del flujo».

Archivo .github/workflows/ci.yml

```yaml
name: CI

on:
  push:
    branches: [ main ]          # ¿en qué rama?
  pull_request:
    branches: [ main ]

permissions:
  contents: read

jobs:
  test:                         # nombre del job (será el check requerido)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: "3.12"
          cache: pip
      - name: Instalar dependencias
        run: pip install -r requirements.txt -r requirements-dev.txt
      - name: Revisar estilo con ruff
        run: ruff check --select E4,E7,E9,F app tests
      - name: Ejecutar pruebas con cobertura
        run: pytest --cov=app --cov-fail-under=80
```

Si su proyecto no usa Python, adapte la instalación y las pruebas a su tecnología conservando la estructura on / jobs / steps.

Evidencia: enlace al pull request del flujo y captura de la pestaña Checks mientras se ejecuta.

Evidencia del equipo: pull request #5 (https://github.com/francito69/SW705U-grupo-A/pull/5), con los espacios del archivo resueltos (en rojo). Primera ejecución: https://github.com/francito69/SW705U-grupo-A/actions/runs/42692833977. Se incluye el registro real de esa ejecución descargado de GitHub Actions. Captura de la pestaña Checks: PENDIENTE (tomarla del PR #5 en el navegador).

Build 1 · registro real de GitHub Actions · ejecución 42692833977 · commit 951c2d4

```
GitHub Actions · flujo «CI» · job «test» · evento pull_request
Ejecución 42692833977 · commit 951c2d4 — ci: agrega flujo de GitHub Actions con ruff y pytest
Resultado: success  ·  https://github.com/francito69/SW705U-grupo-A/actions/runs/42692833977
--------------------------------------------------------------------------------
▶ Set up job ✓
▶ Run actions/checkout@v7 ✓
▶ Run actions/setup-python@v7 (Python 3.12.16) ✓
▶ Instalar dependencias
  $ pip install -r requirements.txt -r requirements-dev.txt
  … (93 líneas de descarga e instalación omitidas)
  Successfully installed annotated-doc-0.0.5 annotated-types-0.8.0 anyio-4.16.0 click-8.5.1 coverage-7.17.2 dnspython-2.9.2 email-va
lidator-2.3.0 fastapi-0.162.2 h11-0.16.0 httpcore2-2.16.0 httpx2-2.16.0 idna-3.20 iniconfig-2.3.1 opentelemetry-api-1.45.2 packaging
-26.3 pluggy-1.7.0 pydantic-2.14.3 pydantic-core-2.50.4 pygments-2.21.0 pytest-9.2.0 pytest-cov-7.1.0 ruff-0.17.4 sqlalchemy-2.1.8 s
tarlette-1.10.0 truststore-0.10.5 typing-extensions-4.16.0 typing-inspection-0.4.5 uvicorn-0.54.0
  ✓ paso correcto
▶ Revisar estilo con ruff
  $ ruff check --select E4,E7,E9,F app tests
  All checks passed!
  ✓ paso correcto
▶ Ejecutar pruebas con cobertura
  $ pytest --cov=app --cov-fail-under=80
  ============================= test session starts ==============================
  platform linux -- Python 3.12.16, pytest-9.2.0, pluggy-1.7.0
  rootdir: /home/runner/work/SW705U-grupo-A/SW705U-grupo-A
  configfile: pyproject.toml
  testpaths: tests
  plugins: cov-7.1.0, anyio-4.16.0
  collected 2 items

  tests/test_base.py ..                                                    [100%]

  ================================ tests coverage ================================
  _____________ coverage: platform linux, python 3.12.16-final-0 _____________

  Name                       Stmts   Miss  Cover
  --------------------------------------------
  app/__init__.py                0      0   100%
  app/crud.py                    6      0   100%
  app/database.py               21      1    95%
  app/main.py                   16      0   100%
  app/models.py                 10      0   100%
  app/routers/__init__.py        0      0   100%
  app/routers/convenios.py       8      0   100%
  app/schemas.py                 7      0   100%
  app/seed.py                   10      1    90%
  --------------------------------------------
  TOTAL                         78      2    97%
  Required test coverage of 80% reached. Total coverage: 97.44%
  ============================== 2 passed in 0.47s ===============================
  ✓ paso correcto
--------------------------------------------------------------------------------
RESULTADO: VERDE ✓
```

Build 1 (commit 951c2d4): registro real de GitHub Actions; resultado verde.

Parte 4 — El primer build rojo y el primer build verde

Lo normal es que la primera ejecución falle. En el código de referencia de la clínica, ruff detecta un import que no se usa en app/routers/citas.py. Aprender a leer un build rojo es el objetivo de esta parte. Si el build de su proyecto salió verde de inmediato, provoque el rojo agregando la línea import os (sin usarla) al inicio de app/main.py.

N.º	Qué hacer	Comando o acción
4.1	Abrir la ejecución fallida en la pestaña Actions, desplegar el paso que falló y copiar el mensaje de error.	GitHub → Actions → ejecución con ✗ → paso "Revisar estilo con ruff"
4.2	Instalar las herramientas y reproducir el error en su equipo con el mismo comando del flujo.	`pip install -r requirements-dev.txt` `ruff check --select E4,E7,E9,F app tests`
4.3	Corregir el problema. En la clínica: quitar models del import de app/routers/citas.py, que queda así:	`from .. import crud, schemas`
4.4	Comprobar localmente que ruff y las pruebas pasan, con los mismos comandos del flujo.	`ruff check --select E4,E7,E9,F app tests` `pytest --cov=app --cov-fail-under=80`

4.5	Registrar la corrección y subirla a la misma rama: el pull request se vuelve a verificar solo.	`git add -A`
		`git commit -m "fix: elimina import sin uso en citas.py"`
		`git push`

4.6	Confirmar en el pull request que el check cambió a verde.	GitHub → pull request → marca verde en Checks

En nuestro caso: el primer build salió verde, así que se provocó el rojo como indica la guía: import os sin usar al inicio de app/main.py (commit 907a84a). El error se reprodujo y corrigió en local (commit 0b0e915).

Registro de builds

N.º	Commit	Resultado	Causa del resultado	Corrección aplicada
1	**`951c2d4`** ci: agrega flujo de GitHub Actions con ruff y pytest	**Verde**	Python 3.12.16 en ubuntu-latest: instalación correcta, ruff «All checks passed!», 2 pruebas pasan y cobertura 97.44 % (mínimo 80 %).	Ninguna. Como salió verde de inmediato, se provocó el rojo según la guía.
2	**`907a84a`** chore: agrega import os sin uso para provocar el build rojo	**Rojo**	Paso «Revisar estilo con ruff»: F401 `os` imported but unused en app/main.py:1:8; «Process completed with exit code 1». El paso de pruebas se omite.	Se reprodujo con ruff check --select E4,E7,E9,F app tests y se eliminó la línea import os.
3	**`0b0e915`** fix: elimina import sin uso en main.py	**Verde**	ruff sin errores; 2 passed; cobertura 97.44 %.	— (build final verde)

Evidencia: captura del registro del build rojo con el error y del build verde final.

Evidencia del equipo: los tres builds corrieron en GitHub Actions dentro del PR #5. Build 1: https://github.com/francito69/SW705U-grupo-A/actions/runs/42692833977 · Build 2 (rojo): https://github.com/francito69/SW705U-grupo-A/actions/runs/42692960329 · Build 3: https://github.com/francito69/SW705U-grupo-A/actions/runs/42693090993. Se incluyen los registros reales de cada ejecución y la reproducción local de los pasos 4.2 a 4.5.

Build 2 · registro real de GitHub Actions · ejecución 42692960329 · commit 907a84a

```
GitHub Actions · flujo «CI» · job «test» · evento pull_request
Ejecución 42692960329 · commit 907a84a — chore: agrega import os sin uso para provocar el build rojo
Resultado: failure  ·  https://github.com/francito69/SW705U-grupo-A/actions/runs/42692960329
--------------------------------------------------------------------------------
▶ Set up job ✓
▶ Run actions/checkout@v7 ✓
▶ Run actions/setup-python@v7 (Python 3.12.16) ✓
▶ Instalar dependencias
  ✓ paso correcto (misma instalación que el build 1, recortada)
▶ Revisar estilo con ruff
  $ ruff check --select E4,E7,E9,F app tests
  F401 [*] `os` imported but unused
   --> app/main.py:1:8
    |
  1 | import os
    |        ^^
  2 | from contextlib import asynccontextmanager
    |
  help: Remove unused import: `os`
    |
    - import os
  1 | from contextlib import asynccontextmanager
    |

  Found 1 error.
  [*] 1 fixable with `--fix`.
  ##[error]Process completed with exit code 1.
  ✗ paso fallido
▶ Ejecutar pruebas con cobertura — omitido
--------------------------------------------------------------------------------
RESULTADO: ROJO ✗
```

Build 2 (commit 907a84a): registro real de GitHub Actions; ruff detecta F401 y el build queda en rojo.

Parte 4 · reproducción local del error, corrección y subida · 9/11/2026

```
$ git pull -q
$ pip install -q -r requirements-dev.txt
$ ruff check --select E4,E7,E9,F app tests
F401 [*] `os` imported but unused
 --> app/main.py:1:8
  |
1 | import os
  |        ^^
2 | from contextlib import asynccontextmanager
  |
help: Remove unused import: `os`
  |
  - import os
1 | from contextlib import asynccontextmanager
  |

Found 1 error.
[*] 1 fixable with `--fix`.
# (corrección: se elimina la línea "import os" de app/main.py)
$ git diff --stat
 app/main.py | 1 -
 1 file changed, 1 deletion(-)
$ ruff check --select E4,E7,E9,F app tests
All checks passed!
$ pytest --cov=app --cov-fail-under=80 -q
..                                                                       [100%]
============================== tests coverage ==============================
_______________ coverage: platform linux, python 3.12.3-final-0 _______________

Name                       Stmts   Miss  Cover
--------------------------------------------
app/__init__.py                0      0   100%
app/crud.py                    6      0   100%
app/database.py               21      1    95%
app/main.py                   16      0   100%
app/models.py                 10      0   100%
app/routers/__init__.py        0      0   100%
app/routers/convenios.py       8      0   100%
app/schemas.py                 7      0   100%
app/seed.py                   10      1    90%
--------------------------------------------
TOTAL                         78      2    97%
Required test coverage of 80% reached. Total coverage: 97.44%
2 passed in 1.35s
$ git add -A
$ git commit -q -m "fix: elimina import sin uso en main.py"
$ git push -q
$ git log --oneline -4
0b0e915 fix: elimina import sin uso en main.py
907a84a chore: agrega import os sin uso para provocar el build rojo
951c2d4 ci: agrega flujo de GitHub Actions con ruff y pytest
baa74cd feat: estructura base del backend FastAPI (HU03)
```

Pasos 4.2 a 4.5: reproducción local del error, corrección, verificación y subida.

Build 3 · registro real de GitHub Actions · ejecución 42693090993 · commit 0b0e915

```
GitHub Actions · flujo «CI» · job «test» · evento pull_request
Ejecución 42693090993 · commit 0b0e915 — fix: elimina import sin uso en main.py
Resultado: success  ·  https://github.com/francito69/SW705U-grupo-A/actions/runs/42693090993
--------------------------------------------------------------------------------
▶ Set up job ✓
▶ Run actions/checkout@v7 ✓
▶ Run actions/setup-python@v7 (Python 3.12.16) ✓
▶ Instalar dependencias
  ✓ paso correcto (misma instalación que el build 1, recortada)
▶ Revisar estilo con ruff
  $ ruff check --select E4,E7,E9,F app tests
  All checks passed!
  ✓ paso correcto
▶ Ejecutar pruebas con cobertura
  $ pytest --cov=app --cov-fail-under=80
  ============================= test session starts ==============================
  platform linux -- Python 3.12.16, pytest-9.2.0, pluggy-1.7.0
  rootdir: /home/runner/work/SW705U-grupo-A/SW705U-grupo-A
  configfile: pyproject.toml
  testpaths: tests
  plugins: cov-7.1.0, anyio-4.16.0
  collected 2 items

  tests/test_base.py ..                                                    [100%]

  ================================ tests coverage ================================
  _____________ coverage: platform linux, python 3.12.16-final-0 _____________

  Name                       Stmts   Miss  Cover
  ----------------------------------------------
  app/__init__.py                0      0   100%
  app/crud.py                    6      0   100%
  app/database.py               21      1    95%
  app/main.py                   16      0   100%
  app/models.py                 10      0   100%
  app/routers/__init__.py        0      0   100%
  app/routers/convenios.py       8      0   100%
  app/schemas.py                 7      0   100%
  app/seed.py                   10      1    90%
  ----------------------------------------------
  TOTAL                         78      2    97%
  Required test coverage of 80% reached. Total coverage: 97.44%
  ============================== 2 passed in 0.60s ===============================
  ✓ paso correcto
--------------------------------------------------------------------------------
RESULTADO: VERDE ✓
```

Build 3 (commit 0b0e915): registro real de GitHub Actions; build final en verde.

Parte 5 — Fusionar el flujo y proteger la rama principal

Con el build en verde, otro integrante revisa el pull request del flujo y lo fusiona. Después la mantenedora activa la protección: la rama principal solo aceptará cambios con pull request, una aprobación y el check de pruebas en verde.

N.º	Qué hacer	Comando o acción
5.1	El revisor abre la pestaña Files changed, revisa el archivo del flujo y aprueba el pull request. Luego se fusiona (Squash and merge) y se borra la rama.	GitHub → pull request → Review changes → Approve → Squash and merge

5.2	Actualizar la copia local.	`git switch main`
		`git pull`
		`git branch -d ci/github-actions`

5.3	Crear la regla de protección de la rama principal.	GitHub → Settings → Rules → Rulesets → New ruleset → New branch ruleset
5.4	Nombrar la regla, activarla (Enforcement: Active) y apuntar a la rama principal.	Target branches → Add target → Include default branch
5.5	Marcar: exigir pull request antes de fusionar (1 aprobación) y exigir que los checks pasen, agregando el check test. Bloquear los push forzados.	Require a pull request before merging Require status checks to pass → Add checks → test Block force pushes

5.6	Probar la protección: intentar un push directo a main y leer el rechazo.	`git switch main`
		`git commit --allow-empty -m "prueba: push directo"`
		`git push`
		`git reset --hard HEAD~1`

En nuestro caso: el repositorio francito69/SW705U-grupo-A es público, así que las reglas de protección están disponibles sin plan de pago. Solo el dueño (Inga Champi) puede crearlas. El check test ya aparece en la lista porque el flujo corrió en el PR #5.

🧭 Si su plan de GitHub no permite reglas en repositorios privados

Las reglas de protección en repositorios privados requieren un plan de pago; en repositorios públicos están disponibles. Con el visto bueno del docente, puede hacer el repositorio público siempre que contenga solo datos ficticios y ningún secreto.

Si no es posible, documente el acuerdo de equipo ("no se fusiona sin check verde y sin aprobación") y adjunte una captura de un pull request que lo cumplió.

El check test solo aparece en la lista después de que el flujo se haya ejecutado al menos una vez.

Configuración de la regla	Valor elegido por el equipo
Nombre de la regla	proteccion-main
Aplicación y rama objetivo	Enforcement: Active · Include default branch (main)
Pull request antes de fusionar	Obligatorio, con 1 aprobación de un integrante distinto del autor
Checks de estado requeridos	test (el job de ci.yml)
Push forzados	Bloqueados (Block force pushes)

Evidencia: captura de la regla configurada y del rechazo del push directo (o el acuerdo de equipo, si aplica la nota anterior).

Evidencia del equipo: valores acordados para la regla. El PR #5 tiene el check test en verde y espera la aprobación de Rashuaman Sapallanay. Fusión, creación de la regla, captura de la regla y del rechazo del push directo (paso 5.6): PENDIENTE.

Parte 6 — Una historia con la prueba escrita primero

Cada integrante repite esta parte con su propia historia. El ejemplo de la clínica es HU-04 Cancelar o reprogramar cita. Primero se traducen los criterios de aceptación a pruebas; las pruebas fallan; después se escribe el código que las hace pasar.

6.1 De los criterios de aceptación a las pruebas

Criterio de aceptación (historia propia)	Prueba automática	Resultado esperado
HU03 E1: con un documento nuevo y los campos obligatorios (nombres, apellidos, fecha de nacimiento, teléfono, correo y convenio), el sistema crea el paciente y lo deja disponible.	`test_registra_paciente_nuevo_con_convenio`	201 con id_paciente; la búsqueda por DNI 70000001 devuelve el mismo paciente.
HU03 E1: el sistema valida el formato del correo y del teléfono.	`test_rechaza_correo_o_telefono_invalido` 2 casos parametrizados: correo sin @ y teléfono de 5 dígitos	422 y el paciente no se crea (la búsqueda responde 404).
HU03 E2: no se permite registrar dos veces el mismo tipo y número de documento.	`test_no_permite_documento_duplicado`	409 con {"mensaje": "Paciente ya registrado", "id_paciente": id del existente} para abrir su ficha.

N.º	Qué hacer	Comando o acción

6.2	Partir de main actualizada (ya con el flujo de CI) y crear la rama de la historia.	`git switch main`
		`git pull`
		`git switch -c feature/HU-04-cancelar-cita`

6.3	Escribir las pruebas en tests/test_citas.py (ver cuadro) y ejecutarlas: debe fallar la del criterio nuevo.	`pytest -q`

6.4	Registrar solo las pruebas, subir la rama y abrir el pull request: el check debe ponerse rojo por la prueba que falla.	`git add tests/test_citas.py`
		`git commit -m "test: cubre cancelacion de citas (HU-04)"`
		`git push -u origin feature/HU-04-cancelar-cita`

6.5	Escribir el código mínimo que hace pasar las pruebas (ver cuadro) y ejecutar pruebas y ruff.	`pytest -q` `ruff check --select E4,E7,E9,F app tests`

6.6	Registrar el código y subirlo: el pull request se vuelve a verificar y debe quedar en verde.	`git add app`
		`git commit -m "feat: impide cancelar citas atendidas (HU-04)"`
		`git push`

6.7	Otro integrante revisa con la lista de verificación de la Semana 9, aprueba y se fusiona (Squash and merge). Borrar la rama.	GitHub → Review changes → Approve → Squash and merge

6.8	Actualizar la copia local y limpiar la rama.	`git switch main`
		`git pull`
		`git branch -d feature/HU-04-cancelar-cita`

En nuestro caso: rama feature/HU03-registro-paciente creada a partir del flujo (0b0e915), porque el PR #5 aún no se fusiona; commit solo de pruebas 42474ea «test: cubre registro y busqueda de pacientes (HU03)» (rojo) y commit del código 6ff250b «feat: registra y busca pacientes con su convenio (HU03)» (verde). Su pull request se abre cuando se fusione el PR #5.

Código de las pruebas (tests/test_citas.py)

Escriba aquí, en su repositorio, las pruebas de los criterios de su historia. Use como modelo las pruebas existentes de tests/ (por ejemplo, test_no_permite_citas_duplicadas_mismo_horario). Una prueba por criterio de aceptación.

Archivo del equipo: tests/test_pacientes.py (commit 42474ea), escrito antes del código.

```
"""HU03 Registro y búsqueda de paciente: una prueba por criterio de aceptación.

Solo datos ficticios (dominio de correo reservado example.com).
"""

import pytest

PACIENTE_FICTICIO = {
    "tipo_documento": "DNI",
    "numero_documento": "70000001",
    "nombres": "Lucía",
    "apellido_paterno": "Prueba",
    "apellido_materno": "Ficticia",
    "fecha_nacimiento": "1990-05-20",
    "sexo": "F",
    "telefono": "987650101",
    "email": "lucia.prueba@example.com",
    "id_convenio": 2,
}


def buscar(client, tipo, numero):
    return client.get(
        "/pacientes", params={"tipo_documento": tipo, "numero_documento": numero}
    )


def test_registra_paciente_nuevo_con_convenio(client):
    # E1: documento nuevo y campos obligatorios completos -> se crea el paciente
    respuesta = client.post("/pacientes", json=PACIENTE_FICTICIO)

    assert respuesta.status_code == 201
    creado = respuesta.json()
    assert creado["id_paciente"] > 0
    assert creado["id_convenio"] == 2

    encontrado = buscar(client, "DNI", "70000001")
    assert encontrado.status_code == 200
    assert encontrado.json()["id_paciente"] == creado["id_paciente"]


@pytest.mark.parametrize(
    "campo, valor_invalido",
    [("email", "lucia.prueba-sin-arroba"), ("telefono", "12345")],
)
def test_rechaza_correo_o_telefono_invalido(client, campo, valor_invalido):
    # E1: el sistema valida el formato del correo y del teléfono
    datos = {**PACIENTE_FICTICIO, campo: valor_invalido}

    respuesta = client.post("/pacientes", json=datos)

    assert respuesta.status_code == 422
    assert buscar(client, "DNI", "70000001").status_code == 404


def test_no_permite_documento_duplicado(client):
    # E2: mismo tipo y número de documento -> 409 con el id del paciente existente
    existente = client.post("/pacientes", json=PACIENTE_FICTICIO).json()

    repetido = client.post(
        "/pacientes", json={**PACIENTE_FICTICIO, "nombres": "Otra Persona"}
    )

    assert repetido.status_code == 409
    assert repetido.json()["detail"] == {
        "mensaje": "Paciente ya registrado",
        "id_paciente": existente["id_paciente"],
    }
```

Resultados de ejecución

Momento	Comando	Resultado
Antes de la historia (rama del flujo en 0b0e915)	`pytest -q`	**2 passed**
Solo con las pruebas (rojo), commit 42474ea	`pytest -q`	**4 failed, 2 passed in 0.33s** Las 4 pruebas de HU03 reciben 404 porque /pacientes aún no existe.
Con el código mínimo (verde), commit 6ff250b	`pytest -q`	**6 passed in 0.33s** Cobertura total 98 % (Parte 7).
Revisión de estilo (ambos commits)	`ruff check --select E4,E7,E9,F app tests`	**All checks passed!**

Lista de verificación del revisor

Pregunta de revisión	Sí / No	Comentario
¿Hay una prueba automática por cada criterio de aceptación?	**Sí**	3 criterios → 3 pruebas; la de formato cubre correo y teléfono con 2 casos.
¿El historial muestra rojo antes del código y verde después?	**Sí**	42474ea (solo pruebas) en rojo → 6ff250b (código) en verde, en local. En el PR de HU03 se confirma en Checks.
¿El check test está en verde y la cobertura es de 80 % o más?	**Sí, local**	98 % en local. El flujo ya corre en GitHub (PR #5); el check de HU03 se confirma al abrir su pull request.
¿Los commits y la rama siguen la convención del equipo?	**Sí**	feature/HU03-registro-paciente; prefijos test:, feat:, ci:, fix: y docs:.
¿Los endpoints están documentados y validan el rol (403)?	**Parcial**	Documentados en /docs con 201, 404, 409 y 422. La validación por rol depende de HU01b (JWT), que aún no existe: queda pendiente para cumplir la DoD.
¿Solo hay datos ficticios y ningún secreto en el cambio?	**Sí**	DNI 70000001 y correos @example.com; la conexión a la BD se lee de DATABASE_URL, sin credenciales en el código.

Nota: *la lista la completó el autor como autoverificación. El revisor (Rashuaman Sapallanay) la confirma o corrige en el pull request antes de aprobar.*

Evidencia: enlace al pull request con el historial de checks (rojo y luego verde) y la aprobación del revisor.

Evidencia del equipo: salidas reales del rojo y del verde y documentación de los códigos de respuesta en /docs. Pull request de HU03 con el historial de checks y la aprobación del revisor: PENDIENTE (se abre al fusionar el PR #5).

Parte 6 · solo las pruebas de HU03 (antes del código) · commit 42474ea · 9/11/2026

```
$ pytest -q
..FFF                                                                    [100%]
================================== FAILURES ===================================
____________________ test_registra_paciente_nuevo_con_convenio ____________________

client = <starlette.testclient.TestClient object at 0x7f68c830b410>

    def test_registra_paciente_nuevo_con_convenio(client):
        # E1: documento nuevo y campos obligatorios completos -> se crea el paciente
        respuesta = client.post("/pacientes", json=PACIENTE_FICTICIO)

>       assert respuesta.status_code == 201
E       assert 404 == 201
E        +  where 404 = <Response [404 Not Found]>.status_code

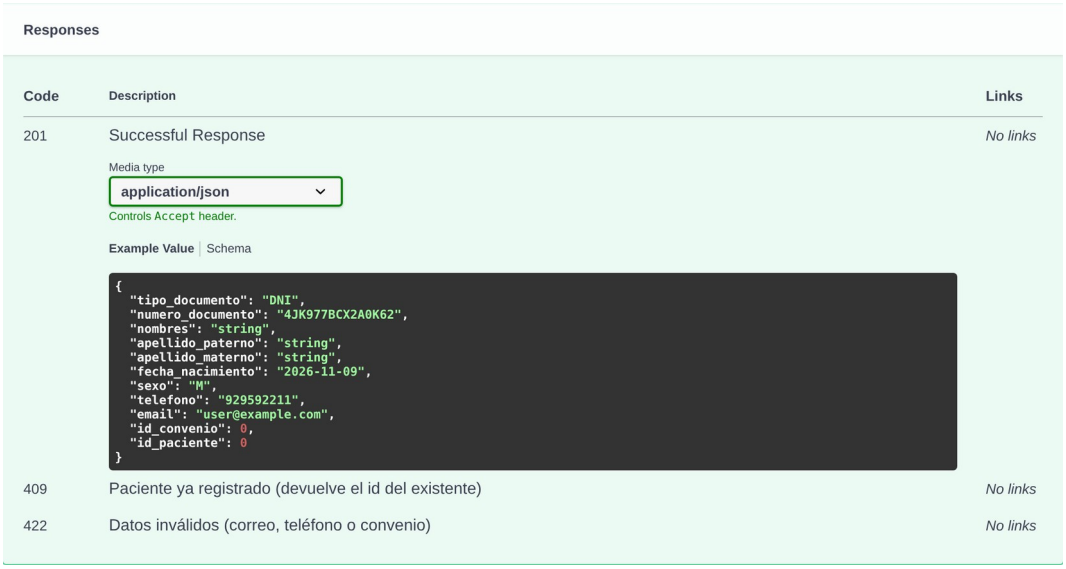
tests/test_pacientes.py:32: AssertionError
… (57 líneas recortadas: las otras 3 fallas también reciben 404 en vez de 422/409)
========================== short test summary info ==========================
FAILED tests/test_pacientes.py::test_registra_paciente_nuevo_con_convenio - a...
FAILED tests/test_pacientes.py::test_rechaza_correo_o_telefono_invalido[email-lucia.prueba-sin-arroba]
FAILED tests/test_pacientes.py::test_rechaza_correo_o_telefono_invalido[telefono-12345]
FAILED tests/test_pacientes.py::test_no_permite_documento_duplicado - assert ...
4 failed, 2 passed in 0.33s
$ ruff check --select E4,E7,E9,F app tests
All checks passed!
$ git add tests/test_pacientes.py
$ git commit -q -m "test: cubre registro y busqueda de pacientes (HU03)" -m "Una prueba por criterio de aceptacion: registro con co
nvenio (E1), validacion de correo y telefono (E1) y documento duplicado 409 (E2)."
$ git log --oneline -1
42474ea test: cubre registro y busqueda de pacientes (HU03)
```

Paso 6.3: con solo las pruebas, las 4 pruebas de HU03 fallan (rojo).

Parte 6 · código mínimo de HU03 · commit 6ff250b · 9/11/2026

```
$ pytest -q
......                                                                   [100%]
6 passed in 0.33s
$ ruff check --select E4,E7,E9,F app tests
All checks passed!
$ git add app
$ git commit -q -m "feat: registra y busca pacientes con su convenio (HU03)" -m "POST /pacientes (201, 409 con el id del existente,
422 por correo, telefono o convenio invalido) y GET /pacientes por tipo y numero de documento (404 si no existe). Modelo Paciente a
lineado con database/01-esquema.sql (uq_paciente_documento)."
$ git log --oneline -3
6ff250b feat: registra y busca pacientes con su convenio (HU03)
42474ea test: cubre registro y busqueda de pacientes (HU03)
0b0e915 fix: elimina import sin uso en main.py
```

Paso 6.5: con el código mínimo pasan las 6 pruebas y ruff no marca errores.



Criterio 5 de la DoD: POST /pacientes documentado con 201, 409 y 422 en /docs.

Historial de commits y ramas de la Semana 10 · 9/11/2026

```
$ git log --oneline origin/main..HEAD
fcb6a97 docs: agrega insignia del build al README
6ff250b feat: registra y busca pacientes con su convenio (HU03)
42474ea test: cubre registro y busqueda de pacientes (HU03)
0b0e915 fix: elimina import sin uso en main.py
907a84a chore: agrega import os sin uso para provocar el build rojo
951c2d4 ci: agrega flujo de GitHub Actions con ruff y pytest
baa74cd feat: estructura base del backend FastAPI (HU03)
$ git branch
  ci/github-actions
* docs/insignia-ci
  feature/HU03-registro-paciente
  main
```

Historial: los 4 commits del PR #5 y los 3 commits de HU03 e insignia, en las ramas de la convención del equipo.

Parte 7 — Cobertura e insignia del build

Un build verde solo es tan confiable como las pruebas que lo respaldan. El flujo ya exige una cobertura mínima de 80 %. Ahora el equipo revisa qué partes del código no tienen pruebas y deja a la vista el estado del build en el README.

N.º	Qué hacer	Comando o acción
7.1	Ejecutar las pruebas mostrando las líneas no cubiertas por módulo.	`pytest --cov=app --cov-report=term-missing`
7.2	Identificar el módulo con menor cobertura y proponer una prueba que lo mejore.	Anotar en la tabla siguiente
7.3	Agregar al inicio del README la insignia del flujo (reemplace USUARIO y REPO).	`![CI](https://github.com/USUARIO/REPO/actions/workflows/ci.yml/badge.svg)`

7.4	Subir el cambio con un pull request pequeño de documentación, siguiendo el mismo proceso.	`git switch -c docs/insignia-ci`
		`git add README.md`
		`git commit -m "docs: agrega insignia del build al README"`
		`git push -u origin docs/insignia-ci`

Módulo	Cobertura	Líneas sin cubrir	Prueba propuesta
`app/seed.py`	90 %	21 (retorno cuando los convenios ya existen)	`test_sembrar_convenios_no_duplica` Llamar dos veces a sembrar_convenios y comprobar que siguen siendo 7 convenios.
`app/routers/pacientes.py`	95 %	49 (convenio inexistente → 422)	`test_rechaza_convenio_inexistente` POST /pacientes con id_convenio = 99 → 422 y la búsqueda responde 404.
`app/database.py`	95 %	24 (engine para PostgreSQL)	`test_engine_postgresql_usa_pool_pre_ping` Reemplazar create_engine con monkeypatch y verificar pool_pre_ping=True con una URL postgresql://.

Las cifras corresponden al código de referencia de la clínica antes de HU-04; las de su proyecto serán distintas.

Cifras de SIRA: después de HU03 (6ff250b): 142 sentencias, 3 sin cubrir, cobertura total 98 %. El módulo con menor cobertura es app/seed.py (90 %).

🧭 Buenas prácticas del flujo

Los secretos (claves de API, contraseñas de base de datos) se guardan en Settings → Secrets del repositorio y nunca en el código ni en el archivo del flujo.

Fije las versiones de las dependencias (requirements.txt y requirements-dev.txt) para que el build de hoy sea igual al de mañana.

Para dar solo los permisos necesarios al flujo, puede agregar al archivo, antes de jobs:, el bloque permissions: con contents: read.

Evidencia: captura del informe de cobertura y de la insignia visible en el README.

Evidencia del equipo: informe de cobertura real y commit de la insignia (rama docs/insignia-ci, commit fcb6a97, por subir). La insignia se verá en el README cuando el flujo esté en main: captura PENDIENTE.

Parte 7 · cobertura con líneas no cubiertas · después de HU03 (6ff250b) · 9/11/2026

```
$ pytest --cov=app --cov-report=term-missing
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.2.0, pluggy-1.7.0
rootdir: /proyectos/SW705U-grupo-A
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.16.0, cov-7.1.0
collected 6 items

tests/test_base.py ..                                                    [ 33%]
tests/test_pacientes.py ....                                             [100%]

================================ tests coverage ================================
_______________ coverage: platform linux, python 3.12.3-final-0 _______________

Name                        Stmts   Miss  Cover   Missing
---------------------------------------------------------
app/__init__.py                 0      0   100%
app/crud.py                    17      0   100%
app/database.py                21      1    95%   24
app/main.py                    17      0   100%
app/models.py                  26      0   100%
app/routers/__init__.py         0      0   100%
app/routers/convenios.py        8      0   100%
app/routers/pacientes.py       20      1    95%   49
app/schemas.py                 23      0   100%
app/seed.py                    10      1    90%   21
---------------------------------------------------------
TOTAL                         142      3    98%
============================== 6 passed in 0.72s ===============================
```

Paso 7.1: pytest --cov=app --cov-report=term-missing.

Parte 7 · insignia del flujo en el README · commit fcb6a97 · 9/11/2026

```
$ git add README.md
$ git commit -q -m "docs: agrega insignia del build al README"
$ git show --stat --format='%h %s' HEAD
fcb6a97 docs: agrega insignia del build al README

 README.md | 1 +
 1 file changed, 1 insertion(+)
$ git show --format= -U1 HEAD -- README.md | cut -c1-110
diff --git a/README.md b/README.md
index 4515acc..edc0d39 100644
--- a/README.md
+++ b/README.md
@@ -2,2 +2,3 @@

+![CI](https://github.com/francito69/SW705U-grupo-A/actions/workflows/ci.yml/badge.svg)
 ![Universidad Nacional de Ingeniería](<https://img.shields.io/badge/Universidad-UNI%20--%20FIIS-blue?style=f
```

Paso 7.3: línea de la insignia agregada al inicio del README.

Parte 8 — Registro del incremento y retrospectiva breve

Al cerrar la jornada, el equipo deja constancia del estado real del Sprint 1: qué historias están hechas según la DoD, cuánto trabajo falta y qué cambia para la Semana 11.

8.1 Estado del Sprint Backlog

Historia	Puntos	Estado	Evidencia (pull request)
HU01a Consulta cronológica de la HCE y HU01b Acceso por rol y bitácora	5 + 3	Por hacer	Sin pull request
HU02 Agendamiento de citas sin solapamientos	5	Por hacer	Sin pull request
HU03 Registro y búsqueda de paciente	3	**En revisión** Estructura base y flujo en el PR #5 con check verde; código y 6 pruebas de HU03 listos. Aún no cumple la DoD: falta PR aprobado, rol 403 (HU01b) y aceptación del PO.	PR #5: https://github.com/francito69/SW705U-grupo-A/pull/5 · rama feature/HU03-registro-paciente por subir
HU04 Atención médica con CIE-10 y receta	5	Por hacer	Sin pull request
HU05 Registro de triaje y signos vitales	3	Por hacer	Sin pull request
HU06 Reprogramación y cancelación de citas	3	Por hacer	Sin pull request

8.2 Avance del Sprint (puntos pendientes)

Corte	Línea ideal	Puntos pendientes reales	Observación
Inicio del Sprint 1 (Semana 9)	27	27	Sprint Backlog del equipo: 27 puntos en 7 historias (la guía usa 24 como referencia).
Fin de la Semana 9	18	27	Ninguna historia en Hecho; el repositorio aún no tenía código.
Semana 10 (corte 9/11/2026)	9 al cierre de la semana	27	HU03 avanza (PR #5 con check verde), pero sin pull request aprobado no descuenta puntos. Desvío frente a la meta de la semana: 18 puntos.
Fin de la Semana 11	0	Por registrar	Riesgo alto: si al cierre de la Semana 10 quedan más de 9 puntos, el PO decide qué historia pasa al Sprint 2.

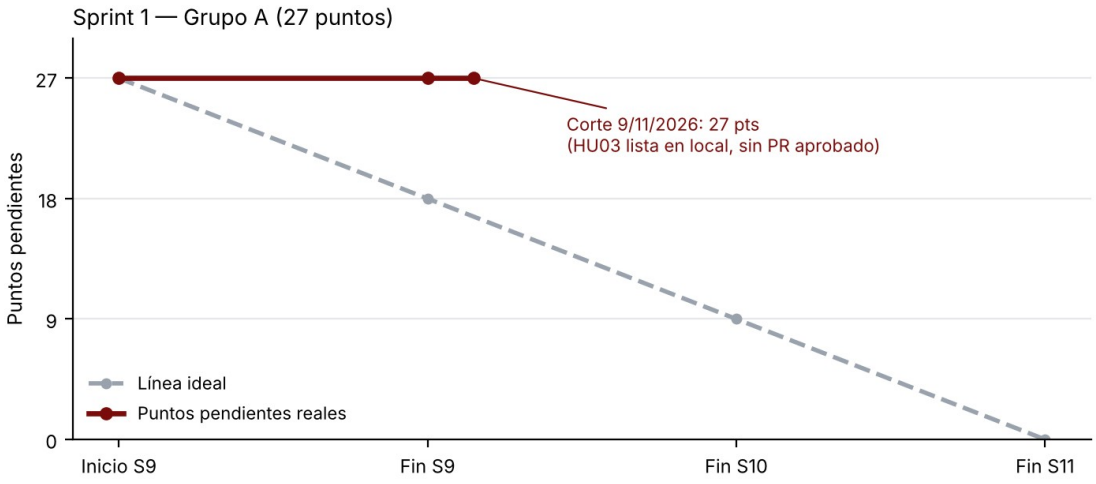
Línea ideal: 24 puntos que bajan en partes iguales hasta 0 al final de la Semana 11. Los puntos pendientes reales se calculan sumando las historias que aún no cumplen la Definición de Hecho.

8.3 Retrospectiva de la jornada

Qué funcionó	Qué mejorar	Acción concreta para la Semana 11 (responsable)
Construir la estructura base con sus pruebas desde el inicio: el flujo salió verde al primer intento en GitHub y la cobertura quedó en 98 %.	Los pull requests dependen uno de otro: el de HU03 no puede abrirse hasta que se fusione el PR #5.	Rashuaman Sapallanay: revisar y aprobar el PR #5 con la lista de verificación (hoy). Albino Soto: abrir el PR de HU03 apenas se fusione (martes 10/11).
Escribir primero las pruebas de HU03 fijó el contrato de la API (201, 409 con el id del existente, 422) antes de programar.	La validación por rol (403) que pide la DoD depende de HU01b, que todavía no existe.	Inga Champi: entregar el inicio de sesión JWT y la dependencia require_role de HU01b para aplicarla a /pacientes (jueves 12/11).
Leer el build rojo (F401 en app/main.py:1) en el registro de Actions y corregirlo con el mismo comando del flujo tomó un solo commit.	La regla de protección de main todavía no existe: hoy se podría fusionar sin check ni aprobación.	Inga Champi: crear la regla proteccion-main al fusionar el PR #5. Rashuaman Sapallanay: aprobar los PR siguientes en menos de 24 h.

Evidencia: capturas del tablero y del gráfico o tabla de avance, y de la retrospectiva.

Evidencia del equipo: tabla de avance y gráfico construidos con los puntos registrados en 8.2; retrospectiva completada. Captura actualizada del tablero: PENDIENTE.



Avance del Sprint 1: línea ideal frente a puntos pendientes reales (corte 9/11/2026).

Anexo — Problemas frecuentes con la integración continua

Síntoma	Causa probable	Qué hacer
El flujo no se ejecuta al subir cambios.	El archivo no está en .github/workflows/ o no termina en .yml; o la rama del push no coincide con la de on:.	Verificar la ruta exacta y los nombres de rama; abrir la pestaña Actions.
Error de sintaxis del flujo (Invalid workflow file).	Sangría incorrecta en el YAML: se usan espacios, nunca tabuladores.	Revisar la línea que indica el mensaje y alinear con 2 espacios por nivel.
Pasa en mi computadora y falla en el servidor.	Falta una dependencia en requirements.txt o requirements-dev.txt, o la versión de Python es distinta.	Instalar en un entorno virtual nuevo con los archivos de requisitos y probar de nuevo.
ModuleNotFoundError al ejecutar pytest.	Dependencia sin declarar o error de importación.	Agregarla al archivo de requisitos con versión fija.
ruff reporta F401 (import sin uso) o F821 (nombre no definido).	Se quitó o se olvidó un import.	Ejecutar el mismo comando de ruff en su equipo y corregir la línea indicada.
Falla por cobertura (Required test coverage of 80% not reached).	Las pruebas no cubren suficiente código.	Ver las líneas sin cubrir con --cov-report=term-missing y agregar pruebas.
El check test no aparece al crear la regla de protección.	El flujo todavía no se ejecutó, o el job tiene otro nombre.	Ejecutar el flujo una vez y usar el nombre exacto del job.
Authentication failed al hacer push.	Token vencido o sin permisos.	Generar un token de acceso personal con permiso sobre el repositorio o usar SSH.
No se pueden crear reglas de protección en el repositorio.	Repositorio privado en un plan que no incluye la función.	Ver la nota de la Parte 5.

Rúbrica de evaluación — Semana 10

Escala vigesimal (0–20). La guía es formativa: la rúbrica orienta la retroalimentación del docente y prepara la calificación sumativa del Avance 5 de la Semana 11. Subir datos reales de pacientes o secretos al repositorio se califica como nivel Inicial en el criterio correspondiente.

Criterio (peso)	Logrado (18–20)	Satisfactorio (14–17)	En proceso (11–13)	Inicial (0–10)
Tablero, asignación y Definición de Hecho (15 %)	Cada integrante tiene una historia real del Sprint Backlog con rama convencional; la DoD incluye el build verde y todos los criterios son verificables.	Asignación completa y DoD con CI; algún criterio es poco verificable.	Asignación o DoD incompletas; la DoD no incorpora la integración continua.	No presenta asignación ni DoD.
Flujo de integración continua (25 %)	Flujo activo en el repositorio con instalación, ruff y pruebas con cobertura mínima; se ejecuta en push y pull request hacia main; el equipo explica cada sección del archivo.	Flujo activo con instalación y pruebas; falta la revisión de estilo o la cobertura mínima; explicación parcial.	Archivo del flujo presente pero con ejecuciones fallidas sin resolver o ejecuta solo parte del proceso.	No hay flujo de integración continua.
Historia con la prueba primero (25 %)	Cada criterio de aceptación tiene una prueba automática; el historial del pull request muestra rojo antes del código y verde después; el código es mínimo y legible.	Pruebas para los criterios principales; el historial de rojo a verde no es del todo claro.	Pruebas que no corresponden a los criterios, o escritas después sin evidenciar la falla inicial.	No hay pruebas automáticas de la historia.
Pull request, revisión y protección (15 %)	Pull request revisado y aprobado por otro integrante con la lista de verificación, fusionado con el check en verde; rama principal protegida (o acuerdo documentado si el plan no lo permite).	Revisión y fusión correctas; la protección está configurada de forma parcial.	Se fusiona sin revisión de otro integrante o con el check en rojo; sin protección ni acuerdo.	Cambios directos a la rama principal.
Diagnóstico del build y cobertura (10 %)	Registro de builds con causa y corrección verificables; identifica el módulo menos cubierto y propone una prueba concreta; la insignia está en el README.	Registro con causa y corrección; análisis de cobertura parcial.	Registro sin causa clara o sin análisis de cobertura.	No documenta los builds.
Registro del incremento y reflexión (10 %)	Estado del Sprint coherente con la DoD, avance calculado correctamente y retrospectiva con una acción concreta, con responsable y plazo.	Estado y avance correctos; la retrospectiva es general.	Estado inconsistente con la DoD o avance mal calculado.	No presenta registro.

Guía de verificación para el docente

Dónde se encuentra la evidencia de cada criterio. El repositorio puede revisarse desde el enlace, sin clonarlo.

Dónde verificar	Qué buscar
Documento y tablero del equipo (criterio 1)	Asignación coherente con el Sprint Backlog; DoD con la condición del build verde.
Repositorio → .github/workflows/ci.yml y pestaña Actions (criterio 2)	Archivo con on, jobs y pasos de instalación, estilo y pruebas; ejecuciones recientes; al menos un rojo y un verde.
Pull request de la historia → Checks y commits (criterio 3)	El primer commit con solo pruebas queda en rojo; el siguiente con el código queda en verde.
Pull request → Reviews y Settings → Rules (criterio 4)	Aprobación de un integrante distinto del autor; regla con el check test requerido.
Registro de builds, informe de cobertura y README (criterio 5)	Causas coherentes con los registros de Actions; insignia visible; módulo menos cubierto identificado.
Tablero, avance y retrospectiva (criterio 6)	Puntos pendientes coherentes con las historias que cumplen la DoD.

Calificación

Criterio	Peso	Nivel alcanzado	Nota (0–20)	Nota ponderada
Tablero, asignación y Definición de Hecho	15 %			
Flujo de integración continua	25 %			
Historia con la prueba primero	25 %			
Pull request, revisión y protección	15 %			
Diagnóstico del build y cobertura	10 %			
Registro del incremento y reflexión	10 %			
NOTA FORMATIVA DE LA SEMANA	100 %			